



智能移动技术 实验报告

课程名称 _____ 智能移动技术

姓 名 _____

日 期 _____ 2025 年 3 月 30 日

指导老师 _____ 王越、熊蓉

一、实验要求

在 matlab 中，利用 ekf_localization.m 文件，实现移动机器人的 EKF 定位算法

- 本任务中的观测模型输入为 GPS 全局绝对观测，运动模型的输入为固定的速度和角速度。运动模型的输入在 robotControl() 函数中已定义，u 包含 V_x, V_y 和角速度 w ，不需要修改。机器人的状态量为三维: $x_{\text{Truth}}, x_{\text{Odom}}, x_{\text{Ekf}}=(x,y,\text{yaw})$ 。
- 在给定的 prepare() 函数中，本程序首先根据机器人速度和运动模型计算定位真值。然后在速度上添加噪声，计算有误差的里程计定位。然后在定位真值上添加噪声，形成 GPS 观测。噪声在文件开始已给定，该函数不需要修改。
- Amination 部分和 finalPlot() 函数亦包含绘图部分，不需要修改。
- 如果需要对变量或者函数功能调整，可自行修改或者优化。

二、原理分析

EKF 算法主要分为预测和更新两个部分

2.1 预测

在预测阶段，EKF 主要完成状态预测和误差协方差矩阵的预测。

1. 状态预测

$$\mathbf{x}_{\text{Pred}} = f(\mathbf{x}_{\text{Ekf}}, \mathbf{u}) \quad (1)$$

其中， $\mathbf{x}_{\text{Pred}}$ 是预测后的状态向量， $f(\cdot)$ 是运动模型函数， $\mathbf{x}_{\text{Ekf}}$ 是上一时刻的状态估计， $\mathbf{u}$ 是控制输入。

2. 雅可比矩阵 J_F 的计算

$$J_F = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{Ekf}}, \mathbf{u}} \quad (2)$$

雅可比矩阵 J_F 是运动模型对状态向量的偏导数。

3. 误差协方差矩阵的预测

$$\mathbf{P}_{\text{Pred}} = J_F \mathbf{P}_{\text{Ekf}} J_F^T + \mathbf{Q} \quad (3)$$

其中， $\mathbf{P}_{\text{Pred}}$ 是预测后的误差协方差矩阵， $\mathbf{P}_{\text{Ekf}}$ 是上一时刻的误差协方差矩阵， $\mathbf{Q}$ 是过程噪声协方差矩阵。

2.2 更新

在更新阶段，EKF 主要完成卡尔曼增益的计算、状态更新和误差协方差矩阵的更新。

- 雅可比矩阵 J_H 的计算

$$J_H = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{Pred}}} \quad (4)$$

雅可比矩阵 J_H 是观测模型对状态向量的偏导数。

- 卡尔曼增益的计算

$$\mathbf{K} = \mathbf{P}_{\text{Pred}} J_H^T (J_H \mathbf{P}_{\text{Pred}} J_H^T + \mathbf{R})^{-1} \quad (5)$$

其中， $\mathbf{K}$ 是卡尔曼增益， $\mathbf{R}$ 是观测噪声协方差矩阵。

- 状态更新

$$\mathbf{x}_{\text{EKF}} = \mathbf{x}_{\text{Pred}} + \mathbf{K}(\mathbf{z} - h(\mathbf{x}_{\text{Pred}})) \quad (6)$$

其中， $\mathbf{z}$ 是观测值， $h(\cdot)$ 是观测模型函数。

- 误差协方差矩阵的更新

$$\mathbf{P}_{\text{EKF}} = (\mathbf{I} - \mathbf{K} J_H) \mathbf{P}_{\text{Pred}} \quad (7)$$

其中， $\mathbf{I}$ 是单位矩阵。

三、具体实现

3.1 初始化

协方差矩阵 convR 和 convQ ，以及其他 EKF 初始化。

```

1 % Simulation parameter
2 global noiseQ
3 noiseQ = diag([0.1 0 degreeToRadian(10)]).^2; %[Vx Vy yawrate]
4
5 global noiseR
6 noiseR = diag([0.5 0.5 degreeToRadian(5)]).^2; %[x y yaw]
7
8 % Covariance Matrix for motion
9 convQ = noiseQ;
10
11 % Covariance Matrix for observation
12 convR = noiseR;
```

3.2 运动模型

运动模型描述了机器人状态的动态变化。状态向量 $\mathbf{x} = [x, y, \theta]'$ ，其中 x 和 y 是机器人的位置坐标， θ 是机器人的朝向（以弧度表示）。控制输入向量 $\mathbf{u} = [v_x, v_y, \omega]'$ ，其

中 v_x 和 v_y 是机器人在局部坐标系下的速度， ω 是机器人的角速度。

运动模型的公式为：

$$\begin{aligned}x_{k+1} &= x_k + v_x \Delta t \cos(\theta_k) - v_y \Delta t \sin(\theta_k), \\y_{k+1} &= y_k + v_x \Delta t \sin(\theta_k) + v_y \Delta t \cos(\theta_k), \\ \theta_{k+1} &= \theta_k + \omega \Delta t,\end{aligned}\tag{8}$$

其中 Δt 是时间步长。

```
1 function x = doMotion(x, u)
2     global dt;
3     x(1) = x(1) + u(1) * dt * cos(x(3)) - u(2) * dt * sin(x(3));
4     x(2) = x(2) + u(1) * dt * sin(x(3)) + u(2) * dt * cos(x(3));
5     x(3) = x(3) + u(3) * dt;
6 end
```

3.3 观测模型

在本实验中，观测模型非常简单，直接将状态向量作为观测值：

$$\mathbf{z} = \mathbf{x}\tag{9}$$

在代码中，观测模型由 `doObservation()` 函数实现。

```
1 function x = doObservation(z, xPred)
2     x = xPred;
3 end
```

3.4 J_F

运动模型的雅可比矩阵 J_F 是运动模型对状态向量的偏导数：

$$J_F = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\mathbf{x}_{\text{EKF}}, \mathbf{u}}\tag{10}$$

具体形式为：

$$J_F = \begin{bmatrix} 1 & 0 & -v_x \Delta t \sin(\theta) - v_y \Delta t \cos(\theta) \\ 0 & 1 & v_x \Delta t \cos(\theta) - v_y \Delta t \sin(\theta) \\ 0 & 0 & 1 \end{bmatrix}\tag{11}$$

在代码中，雅可比矩阵 J_F 由 `jacobF()` 函数计算。

```

1 function jF = jacobF(x, u)
2     global dt;
3     jF = [1, 0, -u(1) * dt * sin(x(3)) - u(2) * dt * cos(x(3));
4           0, 1, u(1) * dt * cos(x(3)) - u(2) * dt * sin(x(3));
5           0, 0, 1];
6 end

```

3.5 J_H

观测模型的雅可比矩阵 J_H 是观测模型对状态向量的偏导数。由于观测模型是单位矩阵，因此：

$$J_H = \mathbf{I} \quad (12)$$

在代码中，雅可比矩阵 J_H 由 `jacobH()` 函数计算。

```

1 function jH = jacobH(x)
2     jH = eye(3);
3 end

```

3.6 预测 (Predict) 阶段

1. 状态预测：

$$\mathbf{x}_{\text{Pred}} = f(\mathbf{x}_{\text{Ekf}}, \mathbf{u}) \quad (13)$$

2. 误差协方差矩阵预测：

$$\mathbf{P}_{\text{Pred}} = J_F \mathbf{P}_{\text{Ekf}} J_F^T + \mathbf{Q} \quad (14)$$

具体代码如下：

```

1 xPred = doMotion(xEkf, u);
2 jF = jacobF(xEkf, u);
3 PPred = jF * PxEkf * jF' + convQ;

```

3.7 更新 (Update) 阶段

1. 卡尔曼增益计算:

$$\mathbf{K} = \mathbf{P}_{\text{Pred}} J_H^T (J_H \mathbf{P}_{\text{Pred}} J_H^T + \mathbf{R})^{-1} \quad (15)$$

2. 状态更新:

$$\mathbf{x}_{\text{Ekf}} = \mathbf{x}_{\text{Pred}} + \mathbf{K}(\mathbf{z} - h(\mathbf{x}_{\text{Pred}})) \quad (16)$$

3. 误差协方差矩阵更新:

$$\mathbf{P}_{\text{Ekf}} = (\mathbf{I} - \mathbf{K} J_H) \mathbf{P}_{\text{Pred}} \quad (17)$$

```
1 jH = jacobH(xPred);
2 K = PPred * jH' / (jH * PPred * jH' + convR);
3 xEkf = xPred + K * (z - doObservation(z, xPred));
4 PxEkf = (eye(size(K * jH)) - K * jH) * PPred;
```

3.8 差值比较

具体代码如下:

```
1 ekf_Error = mean(sqrt(sum((estimation.xEkf - estimation.xTruth).^2,
2    2)));
3 odom_Error = mean(sqrt(sum((estimation.xOdom - estimation.xTruth)
4    .^2, 2)));
5 % 输出误差
6 disp('定位误差统计: ');
7 disp(['Odom平均误差: ', num2str(odom_Error), ' 米']);
8 disp(['EKF平均误差: ', num2str(ekf_Error), ' 米']);
```

四、 结果展示

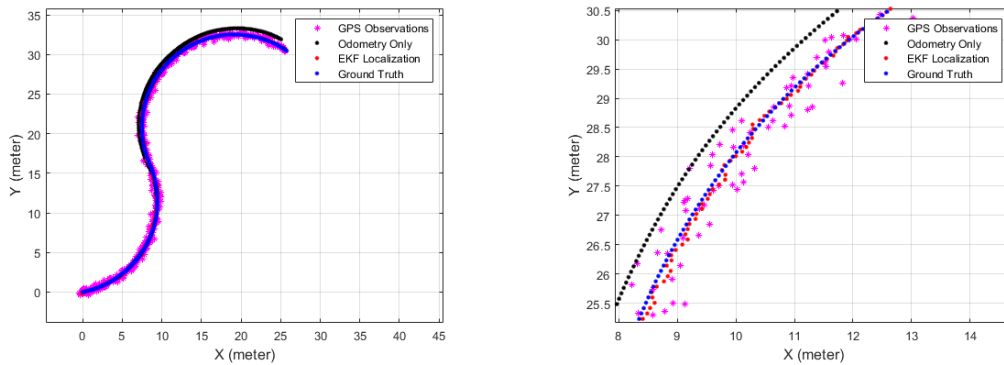
得到的结果如下所示：

```
EKF Start!  
定位误差统计：  
Odom平均误差： 0.41545 米  
EKF平均误差： 0.069047 米
```

图 1 比较结果

在经过多次测量结果后，EKF 平均误差均低于 **0.1m**，且结果均远优于纯里程平均误差。

具体图像如下图所示



五、 结果分析

可以看到 EKF 算法与真值之间还存在一定的误差，可能的原因有以下几个

- 过程噪声的影响：EKF 在预测状态时考虑了过程噪声，即系统动态的不确定性。过程噪声的大小会影响状态的预测精度，通常由噪声协方差矩阵来表示。
- 初始状态和初始协方差的准确性：EKF 需要初始状态和初始状态协方差来开始估计。如果初始状态或初始状态协方差存在误差，那么预测的状态也会有误差，并且这种误差可能会在滤波过程中累积。
- 运动模型的准确性：EKF 中的预测步骤使用了运动模型来预测下一个时刻的状态。如果运动模型与实际系统的运动行为不匹配或者存在误差，那么预测的状态也会有误差。

附录 A 文件列表

文件名	功能描述
data	实验数据与示例
src/result	实验结果
src/icp_map.m	相邻两帧点云匹配与结果绘制
src/traj_map.m	融合点云及轨迹结果和绘制
report.pdf	实验报告
README.md	文件简介